

Compiler Construction

Teacher : Zulfiqar Ali

Email: Zulfiqar.rafique@nu.edu.pk

Office: No.6,Basement -1 (CS Block 1)

Week 6:

- Recursive Decent Parsing
- Non-Recursive Descent Parser - LL(1) Parser

Recursive Descent Parser

- A recursive descent parser is a top-down parser
- It processes input based on a set of recursive functions.
- Where each function corresponds to a grammar rule.
- It parses the input from left to right, constructing a parse tree by matching the grammar's production rules.
- This parser is simple to implement and is suitable for LL(1) grammars.
- where decisions can be made based on a single lookahead token.

Algorithm for Recursive Descent Parser

4.4.1 Recursive-Descent Parsing

-

```
void A() {  
1)    Choose an  $A$ -production,  $A \rightarrow X_1X_2 \cdots X_k$ ;  
2)    for (  $i = 1$  to  $k$  ) {  
3)        if (  $X_i$  is a nonterminal )  
4)            call procedure  $X_i()$ ;  
5)        else if (  $X_i$  equals the current input symbol  $a$  )  
6)            advance the input to the next symbol;  
7)        else /* an error has occurred */;  
    }  
}
```

Figure 4.13: A typical procedure for a nonterminal in a top-down parser

Recursive Descent Parser Pseudocode

```
int parse_P() {  
    return parse_E() && expect_token(TOKEN_EOF);  
}  
  
int parse_E() {  
    return parse_T() && parse_E_prime();  
}  
  
int parse_E_prime() {  
    token_t t = scan_token();  
    if(t==TOKEN_PLUS) {  
        return parse_T() && parse_E_prime();  
    } else {  
        putback_token(t);  
        return 1;  
    }  
}
```

Recursive Descent Parser Pseudocode

```
int parse_T() {  
    return parse_F() && parse_T_prime();  
}  
  
int parse_T_prime() {  
    token_t t = scan_token();  
    if (t==TOKEN_MULTIPLY) {  
        return parse_F() && parse_T_prime();  
    } else {  
        putback_token(t);  
        return 1;  
    }  
}
```

Recursive Descent Parser Pseudocode

```
int parse_F() {  
    token_t t = scan_token();  
    if(t==TOKEN_LPAREN) {  
        return parse_E() && expect_token(TOKEN_RPAREN);  
    } else if(t==TOKEN_INT) {  
        return 1;  
    } else {  
        printf("parse error: unexpected token %s\n",  
               token_string(t));  
        return 0;  
    }  
}
```

Example (Recursive Descent Grammar)

$$E \rightarrow i E'$$
$$E' \rightarrow + i E' \mid \epsilon$$

Function E()

```
E()
{
    if (input == 'i') { // If the input is 'i' (identifier)
        input++;        // Consume 'i'
    }
    E'(); // Call E' to check for further expressions
}
```


Function $E'()$

```
void E`() {  
    if (input == '+') {  
        input++;           // Consume the '+'  
  
        if (input == 'i') {  
            input++;       // Consume the 'i'  
        }  
  
        E`();              // Recursively process more additions  
    } else {  
        return;           // If no '+', return ( $\epsilon$  production)  
    }  
}
```

Main Function

```
Main()
{
    E();          // Start parsing from E
    if (input == '$') // If we reach end of input
        Parsing Successful;
}
```

- Calls $E()$ to start parsing.
- Checks if the input ends with \$, which indicates a successful parse.

Example Input Parsing

Let's consider the example input:

```
i + i $
```

Processing step by step:

1. $E()$ starts $\rightarrow$ input == i, so consume i
2. Call $E'()$ $\rightarrow$ input == +, so consume +
3. input == i, so consume i
4. Call $E'()$ again $\rightarrow$ no +, so return.
5. Back to $Main()$, input == \$ $\rightarrow$ Parsing Successful

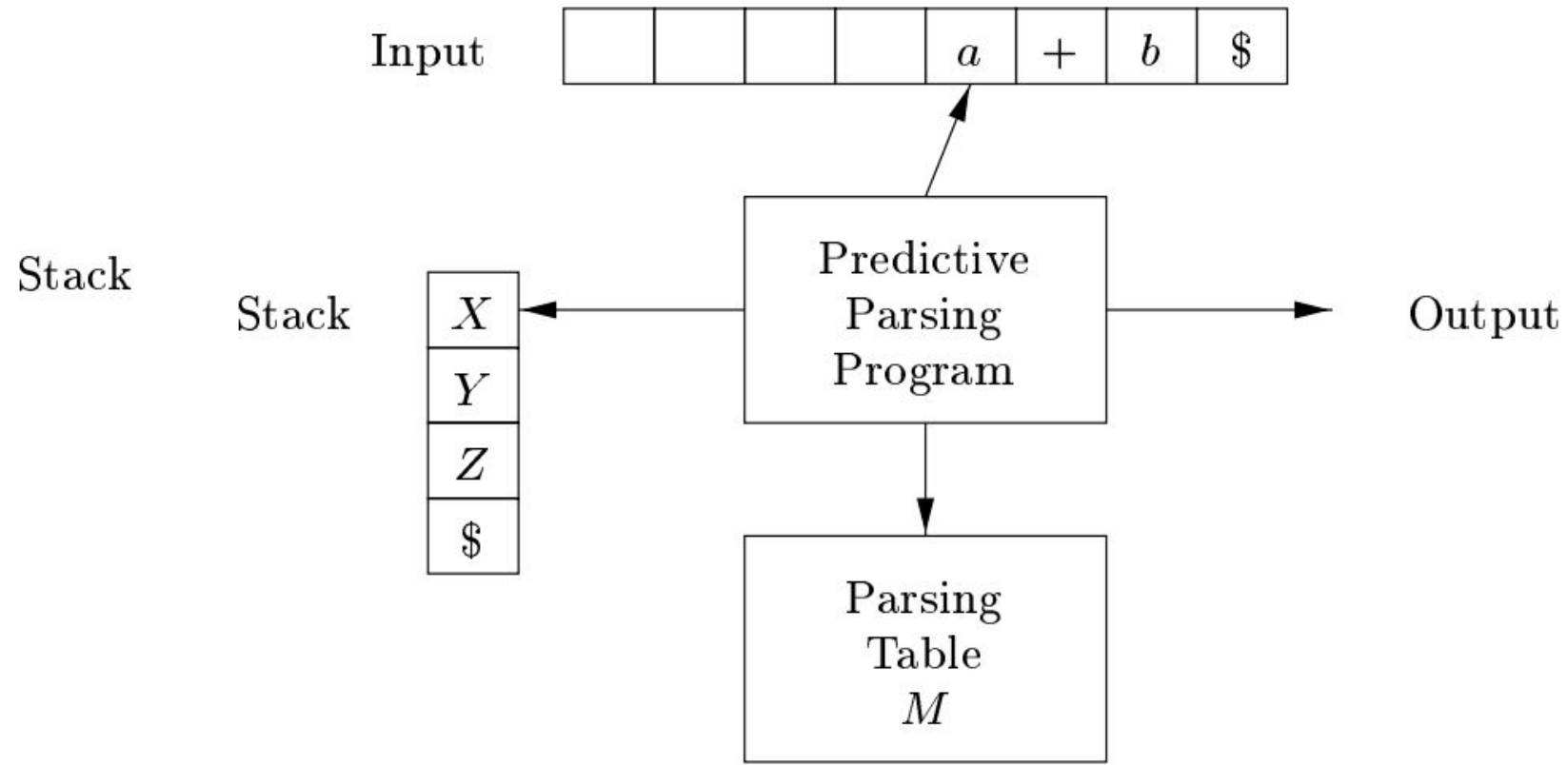
Non-Recursive Descent Parser (Table Driven)/Predictive Parser

- This type of parser is more often generated by a parser generator (like yacc or bison).
- It uses two main data structures:
- An **input buffer** containing the tokens to be parsed.
- A **stack** (which you manage in your code) that holds grammar symbols (both terminals and non-terminals).
- A **parsing table** (a 2D array) that tells the parser what grammar rule to apply based on the current symbol on the stack and the next input token.
- The grammar should be passed by LL(1) grammar

Algorithm 4.34: Table-driven predictive parsing.

INPUT: A string w and a parsing table M for grammar G .

OUTPUT: If w is in $L(G)$, a leftmost derivation of w ; otherwise, an error indication.



Fig

Figure 4.19: Model of a table-driven predictive parser

Non-Recursive Descent – Method(function)

```
let  $a$  be the first symbol of  $w$ ;  
let  $X$  be the top stack symbol;  
while (  $X \neq \$$  ) { /* stack is not empty */  
    if (  $X = a$  ) pop the stack and let  $a$  be the next symbol of  $w$ ;  
    else if (  $X$  is a terminal ) error();  
    else if (  $M[X, a]$  is an error entry ) error();  
    else if (  $M[X, a] = X \rightarrow Y_1 Y_2 \cdots Y_k$  ) {  
        output the production  $X \rightarrow Y_1 Y_2 \cdots Y_k$ ;  
        pop the stack;  
        push  $Y_k, Y_{k-1}, \dots, Y_1$  onto the stack, with  $Y_1$  on top;  
    }  
    let  $X$  be the top stack symbol;  
}
```

This algorithm simulates a pushdown automaton. It uses:

1. An **input string** w (the code to parse).
2. A **stack** to keep track of what we're trying to match.
3. A **parsing table** M that tells the parser which grammar rule to apply based on the top of the stack and the next input symbol.

LL(1) Parsing

- LL(1) Parsing: Here the 1st L represents that the scanning of the Input will be done from the Left to Right manner and the second L shows that in this parsing technique, we are going to use the Left most Derivation Tree. And finally, the 1 represents the number of look-ahead, which means how many symbols are you going to see when you want to make a decision.

Essential conditions for LL(1)

- The grammar is free from left recursion.
- The grammar should not be ambiguous.
- The grammar has to be left factored in so that the grammar is deterministic grammar.
- Find First() and Follow() of the Grammar
- Creating LL(1) Parsing Table

Algorithm to construct LL(1) Parsing Table

- **Step 1:** First check all the essential conditions mentioned above and go to step 2.
- • **Step 2:** Calculate First() and Follow() for all non-terminals.
- • **Step 3:** Make a parser table.
- • **Step 4:** Illustrate LL(1) Parse Tree

LL(1) Example 1

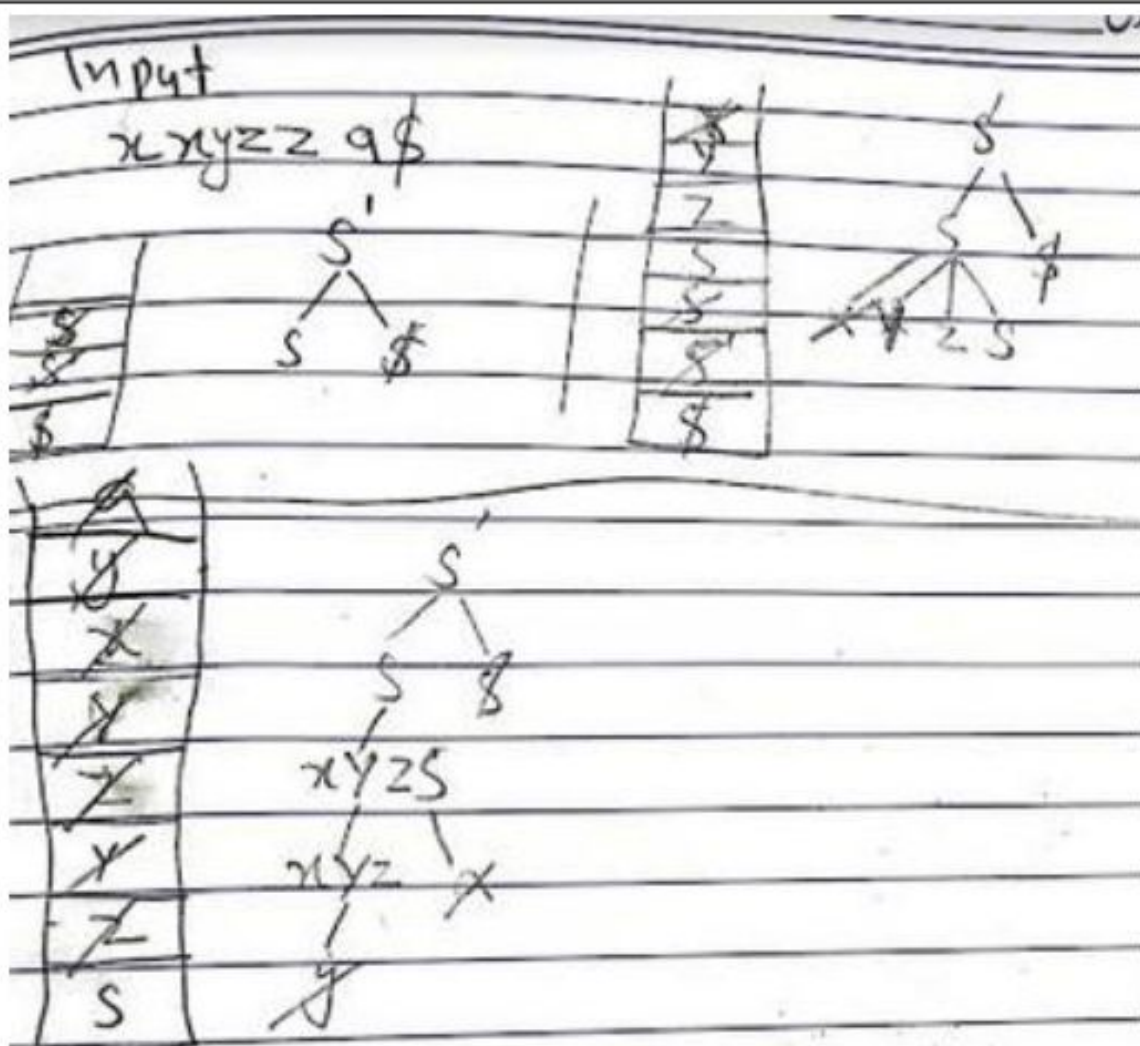
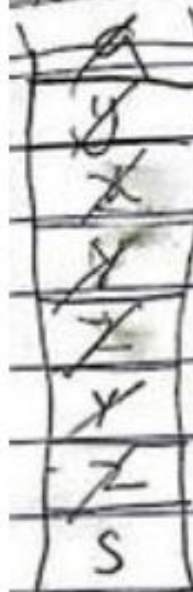
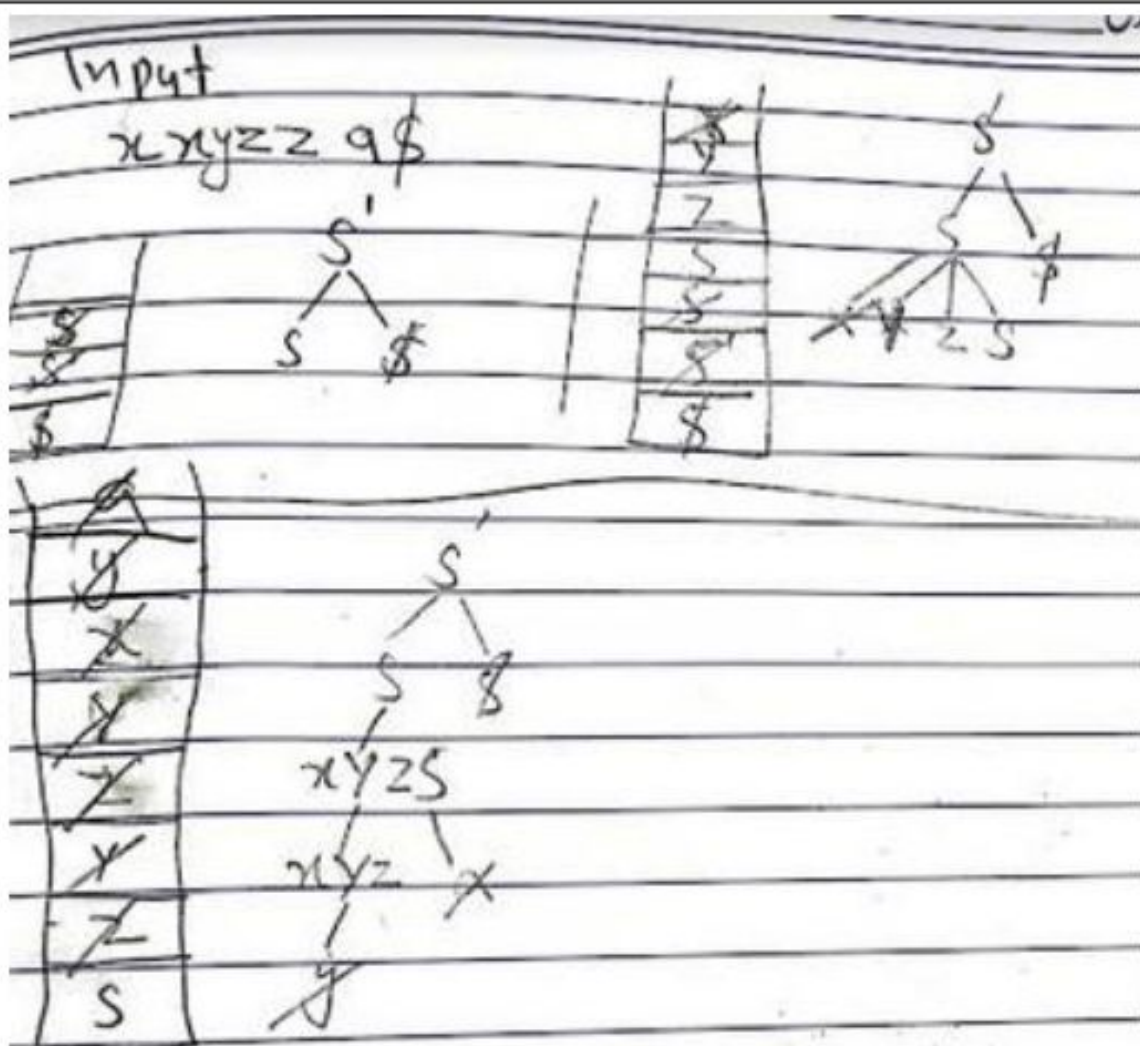
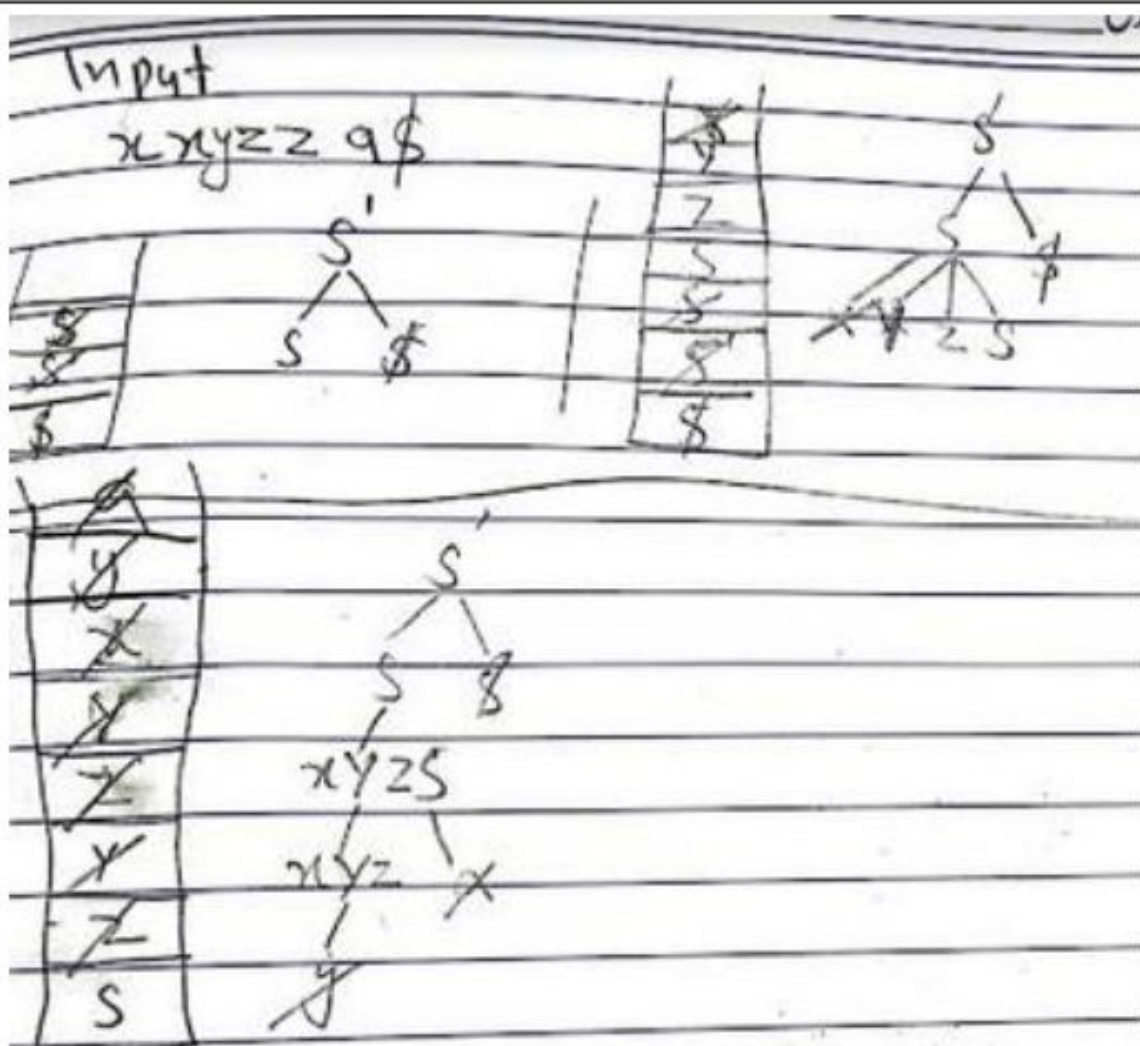
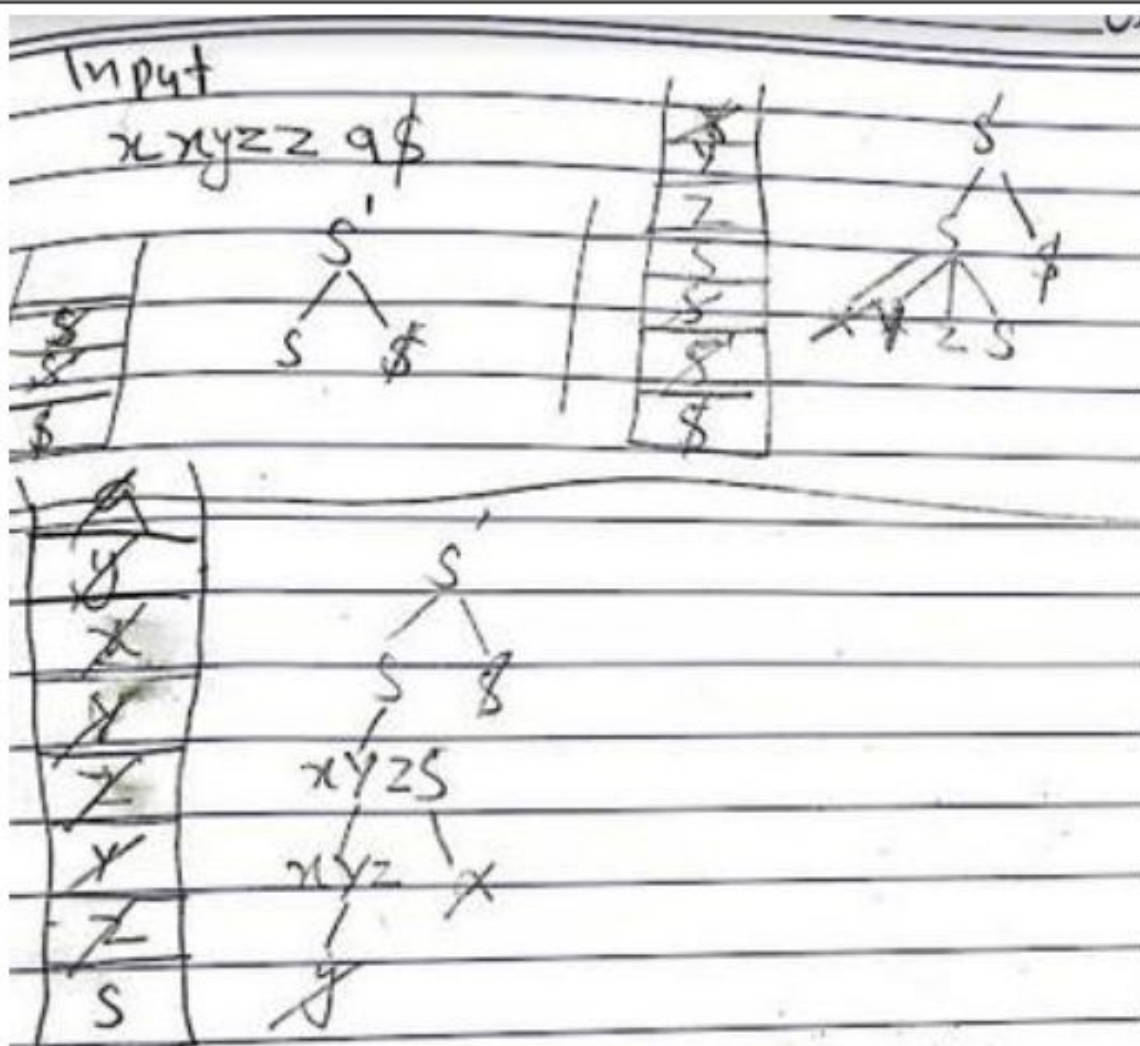
- Let the grammar
- – $S' \rightarrow S\$$
- $S \rightarrow xYzS \mid a$
- $Y \rightarrow xYz \mid y$
- • This grammar is satisfying conditions, Not-Left Recursive, Left Factored,
- we will need to find First() and Follow() and then Parse Table

- $S' \rightarrow S\$$
- $S \rightarrow \underline{xYz}S \mid a$
- $Y \rightarrow \underline{xYz} \mid y$

	First()	Follow()
S'	{x, a}	{\\$}
S	{x, a}	{\\$}
Y	{x, y}	{z}

- LL(1) Parsing Table

	a	x	y	z	\$
S'	$S' \rightarrow S\$$	$S' \rightarrow S\$$			
S	$S \rightarrow a$	$S \rightarrow xYzS$			
Y		$Y \rightarrow xYz$	$Y \rightarrow y$		

[illegible][illegible]

LL(1) Parsing Example 2

- Consider we have Grammar
 - $S \rightarrow aABb$
 - $A \rightarrow c \mid \epsilon$
 - $B \rightarrow d \mid \epsilon$
- This grammar is Right Recursive, and not need to left factorization.

– $S \rightarrow aABb$

$A \rightarrow c \mid \epsilon$

$B \rightarrow d \mid \epsilon$

Find its First() and Follow()

	First	Follow
S	a	\$
A	c, ϵ	d, b
B	d, ϵ	b

	a	b	c	d	\$
S	$S \rightarrow aABb$				
A		$A \rightarrow \epsilon$	$A \rightarrow c$	$A \rightarrow \epsilon$	
B		$B \rightarrow \epsilon$		$B \rightarrow d$	

	a	b	c	d	\$
S	S→aABb				
A		A→ε	A→c	A→ε	
B		B→ε		B→d	

Stack	Input	Action
\$	acdb\$	Push S into Stack
\$S	acdb\$	S→aABb
\$bBAa	acdb\$	Pop a
\$bBA	cdb\$	A→c
\$bBc	cdb\$	Pop c
\$bB	db\$	B→d
\$bd	db\$	Pop d
\$b	b\$	Pop b
\$	\$	Accept

Example 2 LL(1)

- Consider the Grammar
 - $S \rightarrow (L) \mid a$
 - $L \rightarrow SL'$
 - $L' \rightarrow)SL' \mid \epsilon$
- Here initial conditions are true, that it is Right Recursive, Left Factored, we need to find First() and follow()

First() and Follow()

	First	Follow
S	(, a	\$,)
L	(, a	)
L'	), ϵ	)

Creating LL(1) Parse Table

	(	)	a	\$
S	$S \rightarrow (L)$		$S \rightarrow a$	
L	$L \rightarrow SL'$		$L \rightarrow SL'$	
L'		$L' \rightarrow (SL'$ $L' \rightarrow \epsilon$		

Its Grammar is satisfies all condntions but still its LL(1) parse tree can't be illustrated because there multiple Production is same cell.

LL(1) Example 3

- Consider the Grammar
 - $S \rightarrow A \mid a$
 - $A \rightarrow a$
- Above grammar is satisfying initial conditions
- Here are the first and follow

	First	Follow
$S \rightarrow A/a$	{ a }	{ \$ }
$A \rightarrow a$	{ a }	{ \$ }

LL(1) Parsing Table

	a	\$
S	$S \rightarrow A, S \rightarrow a$	
A	$A \rightarrow a$	

LL(1) Example 4

- Consider the Grammar
 - $E \rightarrow TE'$
 - $E' \rightarrow +TE' \mid \epsilon$
 - $T \rightarrow FT'$
 - $T' \rightarrow *FT' \mid \epsilon$
 - $F \rightarrow id \mid (E)$
- This Grammar is satisfying all the condition
- Now Find First() and Follow()

Calculate first() and follow().

	First	Follow
$E \rightarrow TE'$	{ id, (}	{ \$,) }
$E' \rightarrow +TE' / \epsilon$	{ +, ϵ }	{ \$,) }
$T \rightarrow FT'$	{ id, (}	{ +, \$,) }
$T' \rightarrow *FT' / \epsilon$	{ *, ϵ }	{ +, \$,) }
$F \rightarrow id/(E)$	{ id, (}	{ *, +, \$,) }

Creating Parse Tree

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

MATCHED	STACK	INPUT	ACTION
	$E\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	
	$TE'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $E \rightarrow TE'$
	$FT'E'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $T \rightarrow FT'$
	$\mathbf{id} T'E'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $F \rightarrow \mathbf{id}$
$\mathbf{id}$	$T'E'\$$	$+ \mathbf{id} * \mathbf{id}\$$	match $\mathbf{id}$
$\mathbf{id}$	$E'\$$	$+ \mathbf{id} * \mathbf{id}\$$	output $T' \rightarrow \epsilon$
$\mathbf{id}$	$+ TE'\$$	$+ \mathbf{id} * \mathbf{id}\$$	output $E' \rightarrow + TE'$
$\mathbf{id} +$	$TE'\$$	$\mathbf{id} * \mathbf{id}\$$	match $+$
$\mathbf{id} +$	$FT'E'\$$	$\mathbf{id} * \mathbf{id}\$$	output $T \rightarrow FT'$
$\mathbf{id} +$	$\mathbf{id} T'E'\$$	$\mathbf{id} * \mathbf{id}\$$	output $F \rightarrow \mathbf{id}$
$\mathbf{id} + \mathbf{id}$	$T'E'\$$	$* \mathbf{id}\$$	match $\mathbf{id}$
$\mathbf{id} + \mathbf{id}$	$* FT'E'\$$	$* \mathbf{id}\$$	output $T' \rightarrow * FT'$
$\mathbf{id} + \mathbf{id} *$	$FT'E'\$$	$\mathbf{id}\$$	match $*$
$\mathbf{id} + \mathbf{id} *$	$\mathbf{id} T'E'\$$	$\mathbf{id}\$$	output $F \rightarrow \mathbf{id}$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$T'E'\$$	$\$$	match $\mathbf{id}$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$E'\$$	$\$$	output $T' \rightarrow \epsilon$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$\$$	$\$$	output $E' \rightarrow \epsilon$

Difference Between Recursive & Non-Recursive. (Top Down Parsers)

Recursive Predictive Descent Parser	Non-Recursive Predictive Descent Parser
It is a technique which may or may not require backtracking process.	It is a technique that does not require any kind of backtracking.
It uses procedures for every non-terminal entity to parse strings.	It finds out productions to use by replacing input string.
It is built from a set of mutually recursive procedures where each procedure implements one of non-terminal s of grammar.	It is a type of top-down approach, which is also a type of recursive parsing that does not uses technique of backtracking.
It contains several small functions one for each non- terminals in grammar.	It uses a look ahead pointer which points to next input symbols to make it parser back tracking free, it puts some constraints on grammar.
It accepts all kinds of grammars.	It accepts LL(k) grammar type of grammar

4.4.5 Error Recovery in Predictive Parsing

This discussion of error recovery refers to the stack of a table-driven predictive parser, since it makes explicit the terminals and nonterminals that the parser hopes to match with the remainder of the input; the techniques can also be used with recursive-descent parsing.

An error is detected during predictive parsing when the terminal on top of the stack does not match the next input symbol or when nonterminal A is on top of the stack, a is the next input symbol, and $M[A, a]$ is **error** (i.e., the parsing-table entry is empty).

Two Types of Errors Handling in Predictive Parsing

- Panic Mode
- Phrase Mode

Panic Mode

- Panic mode is the skipping over symbols on the input until a token in a selected set of synchronizing tokens appears.
- effectiveness depends on the choice of synchronizing set.
- The sets should be chosen so that the parser recovers quickly from errors that are likely to occur in practice. Some heuristics are as follows:
 - Pop Non-terminal from the Stack
 - Discard/remove terminal from the string
 -

Panic mode :

1. Pop non terminal from the stack
2. discard (remove) the terminal from the string
- X 3. Introduce the nonterminal by which that terminal can be produced. (X).

Parsing stack

T
E'
)
T'
E'
\$

string
*) \$

	(	)	+	-	*	/	↑	id	\$
E	$E \rightarrow TE'$	Pop	Scan	Scan	Scan	Scan	Scan	$E \rightarrow TE'$	Pop
E'	Scan	$E' \rightarrow \epsilon$	$E' \rightarrow +TE'$	$E' \rightarrow -TE'$	Scan	Scan	Scan	Scan	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$	Pop	Pop	Pop	Scan	Scan	Scan	$T \rightarrow FT'$	Pop
T'	Scan	$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$	$T' \rightarrow /FT'$	Scan	Scan	$T' \rightarrow \epsilon$
F	$F \rightarrow GG'$	Pop	Pop	Pop	Pop	Pop	Scan	$F \rightarrow GG'$	Pop
G'	Scan	$G' \rightarrow \epsilon$	$G' \rightarrow \epsilon$	$G' \rightarrow \epsilon$	$G' \rightarrow \epsilon$	$G' \rightarrow \epsilon$	$G' \rightarrow \uparrow F$	Scan	$G' \rightarrow \epsilon$
G	$G \rightarrow (E)$	Pop	Pop	Pop	Pop	Pop	Pop	$G \rightarrow id$	Pop

Reference:

- Compilers: Principles, Techniques, and Tools 2nd Edition
by Alfred Aho , Jeffrey Ullman, Ravi Sethi, Monica Lam
- Introduction to Compilers and Language Design | Second Edition |
Prof. Douglas Thain - University of Notre Dame